Mini Juego "Piedra, Papel o Tijera" con Persistencia de Datos

Actividad 7 - Laboratorio de Programación

E.E.S.T. N° 1 "Eduardo Ader" - Vicente López

Autor: Malek Cheheid - Grupo A

Docente: Yamil Ganduglia

16 de octubre de 2026

GitHub: https://github.com/malekcheheid-art/piedrapapelotijera

GitLab: https://gitlab.com/malekcheheid-art/piedrapapelotijera

Introducción

En esta actividad desarrollé un mini juego de Piedra, Papel o Tijera que se ejecuta en la consola de Windows y guarda cada partida en un archivo de texto. El objetivo principal fue comprender la diferencia entre la memoria RAM, que se borra cuando termina el programa, y el almacenamiento en disco, que conserva los datos de forma permanente. Organicé el código en funciones claras y usé una estructura Partida para agrupar toda la información de una jugada. También preparé un repositorio en GitHub y en GitLab con la estructura requerida, capturas de pantalla, informes y manuales.

Qué hice en el programa

Al iniciar, el programa muestra un menú con tres opciones: jugar una partida, ver el historial guardado y salir. Cuando el usuario elige jugar, el programa pide el nombre del jugador y su elección. La inteligencia artificial elige una opción al azar usando rand(). Después evalúa las reglas del juego y muestra el resultado. Antes de guardar, valido que el nombre no esté vacío y que la opción de jugada sea un número entre 1 y 3, repitiendo la pregunta si es necesario. Para persistir los datos uso la biblioteca fstream y abro el archivo en modo append, por lo que cada nueva partida se agrega al final sin borrar las anteriores.

La estructura struct

Definí una estructura llamada Partida con los campos idPartida, jugador, eleccionJugador, eleccionIA y resultado. Usar struct me permitió tratar toda esa información como una sola entidad. En lugar de pasar cinco variables sueltas a cada función, paso un solo objeto Partida. Eso hace el código más ordenado y más fácil de mantener. En proyectos más grandes esta técnica sirve para registrar muchos datos distintos bajo una misma entidad, por ejemplo lecturas de sensores o eventos de usuarios.

Escritura y lectura en disco

Para escribir uso ofstream con la bandera ios::app. Esto indica al sistema operativo que agregue los nuevos datos al final del archivo sin sobrescribir lo existente. Para leer uso ifstream y recorro todos los registros separando los campos por comas, ya que guardé los datos en formato CSV. Si el archivo no existe, el programa lo crea automáticamente la primera vez que se juega. Si el archivo está vacío o no se puede abrir, muestro un aviso en lugar de dar un error fatal. Con esto logré que la opción 2 recupere y muestre todas las partidas guardadas, incluso después de cerrar y volver a abrir la consola.

Manejo del búfer de entrada

Antes de usar getline para leer el nombre del jugador, llamo a cin.ignore(). Esto es necesario porque el operador cin deja el carácter Enter pendiente en el búfer, y si no limpio ese búfer, getline lee una cadena vacía y se salta la pregunta. Además, manejo posibles errores de entrada numérica con cin.clear() e ignore, para que el programa no entre en un bucle infinito si el usuario escribe letras en lugar de números.

IDs persistentes

Un detalle que agregué por mi cuenta es que el contador de IDs no reinicia en 1 cada vez que abro el programa. Implementé la función cargarContadorID, que lee el archivo, busca el ID más alto y continúa desde ahí. Así, aunque cierre la consola, las nuevas partidas reciben IDs consecutivos y no se repiten. Esto hace que el historial sea coherente en el tiempo.

Implementación de la elección de la IA

Uso srand(time(0)) al inicio de main para inicializar la semilla de números aleatorios. Luego, en cada partida, genero un número entre 1 y 3 con (rand() % 3) + 1 y lo convierto en

Piedra, Papel o Tijera. La evaluación de las reglas se hace comparando cadenas, lo que deja el código fácil de leer y mantener.

Evidencias de ejecución

A continuación muestro capturas reales del programa en funcionamiento. Las dos primeras corresponden a la partida jugada en la consola y al archivo de texto abierto, donde se ven los registros persistidos.

Figura 1. Partida jugada en la consola.

```
PS C:\Users\Malek\OneDrive\Desktop\piedrapapelotijera> .\src\piedrapapelotijera.exe
==================================================
 LPR 2026 - ACTIVIDAD 7: PIEDRA, PAPEL O TIJERA
 Estudiante / Grupo: Malek Cheheid (Grupo A)
==================================================

--- MENU PRINCIPAL DE JUEGO Y PERSISTENCIA ---
1. Jugar Partida (Piedra, Papel o Tijera)
2. Ver Historial de Partidas Guardadas en Disco (.txt)
3. Salir del Sistema
Ingrese una opcion (1-3): 1

--- NUEVA PARTIDA #2 ---
Ingrese su Nombre / Nickname: malek

Elija su jugada:
1. Piedra
2. Papel
3. Tijera
Opcion (1-3): 2

[JUGADA] malek eligio: Papel
[JUGADA] La IA eligio: Piedra
=> RESULTADO: Victoria!
[DISCO] Partida resguardada con exito en 'historial_partidas.txt'.

--- MENU PRINCIPAL DE JUEGO Y PERSISTENCIA ---
1. Jugar Partida (Piedra, Papel o Tijera)
2. Ver Historial de Partidas Guardadas en Disco (.txt)
3. Salir del Sistema
Ingrese una opcion (1-3): 2

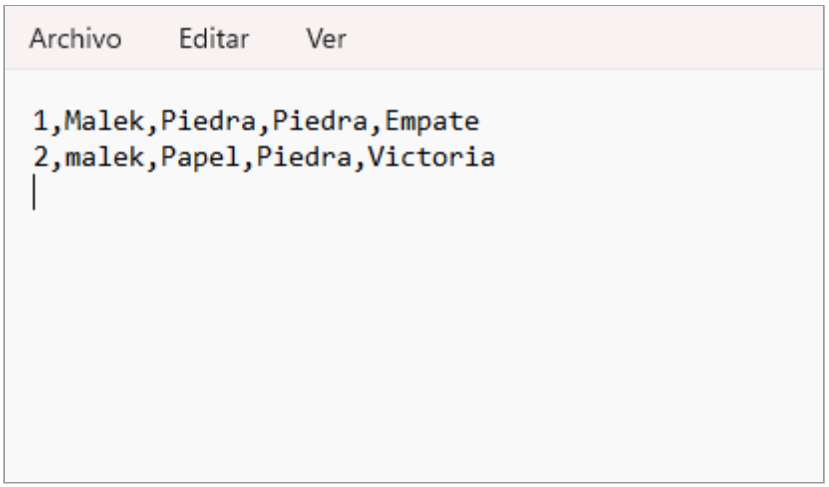
==================================================
 HISTORIAL DE PARTIDAS RECUPERADO DESDE DISCO
==================================================
Partida #1 | Jugador: Malek | Piedra vs Piedra | Resultado: Empate
Partida #2 | Jugador: malek | Papel vs Piedra | Resultado: Victoria

[SISTEMA] Total de registros leidos desde disco: 2
==================================================

--- MENU PRINCIPAL DE JUEGO Y PERSISTENCIA ---
1. Jugar Partida (Piedra, Papel o Tijera)
2. Ver Historial de Partidas Guardadas en Disco (.txt)
3. Salir del Sistema
Ingrese una opcion (1-3): 3

[SISTEMA] Saliendo del programa... Datos resguardados en disco.
PS C:\Users\Malek\OneDrive\Desktop\piedrapapelotijera>
```

Figura 2. Archivo historial_partidas.txt con los registros persistidos.



Sincronización Git Dual-Remote

El código fuente y las capturas se subieron a GitHub y a GitLab mediante la secuencia de dual push. Las siguientes capturas muestran el proyecto publicado en cada plataforma y la verificación de que ambos remotes apuntan al mismo commit.

Figura 3. Repositorio publicado en GitHub.

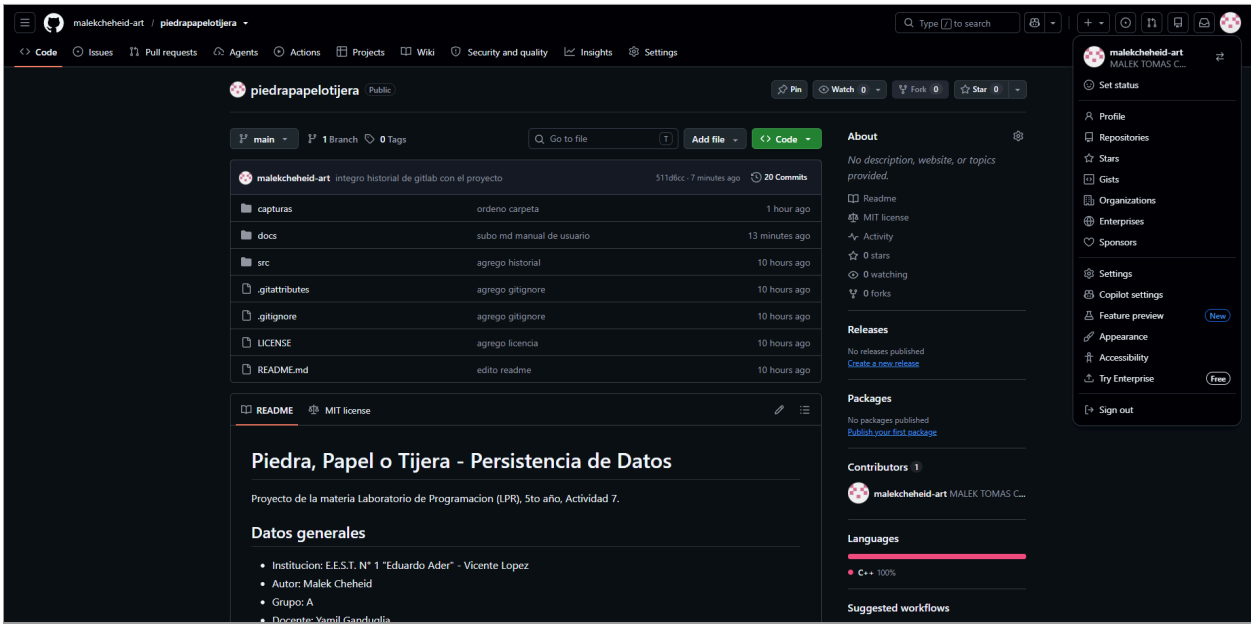


Figura 4. Repositorio publicado en GitLab.

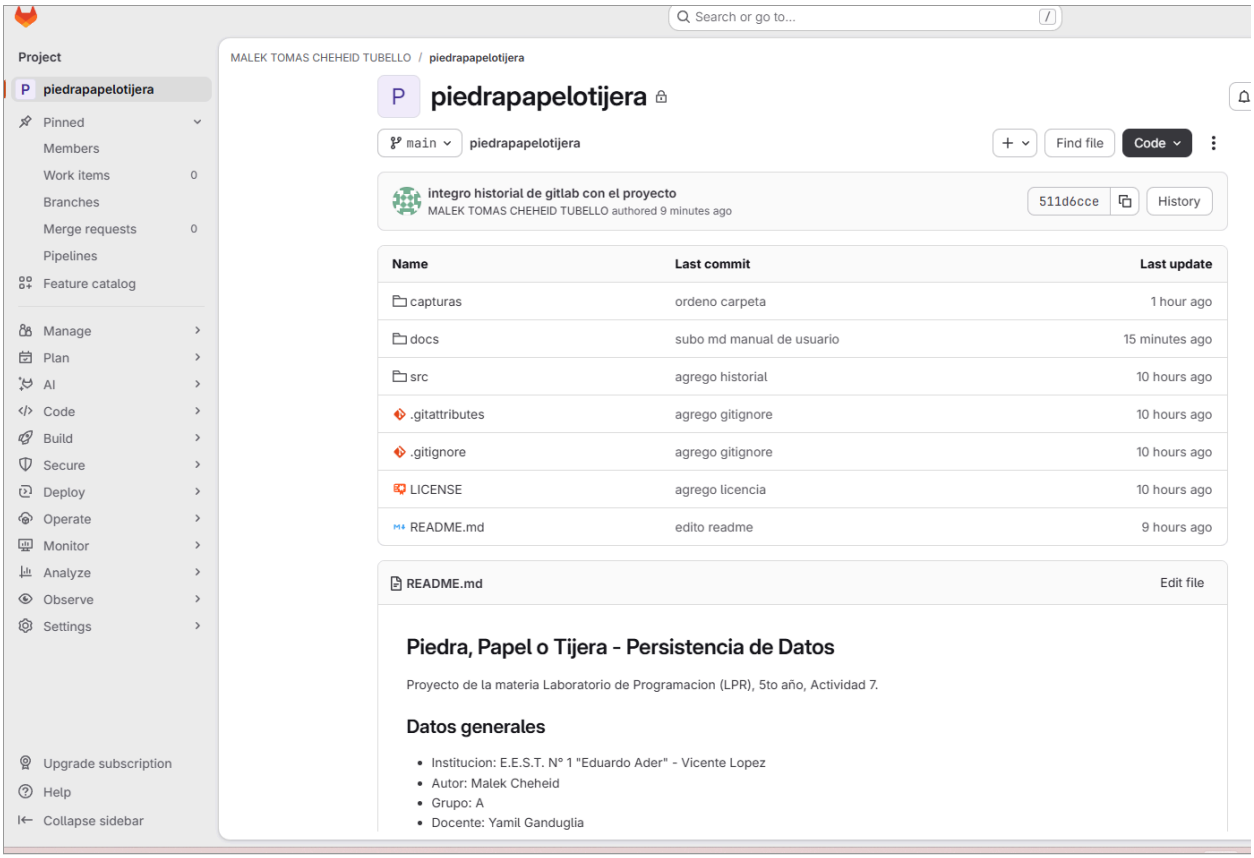
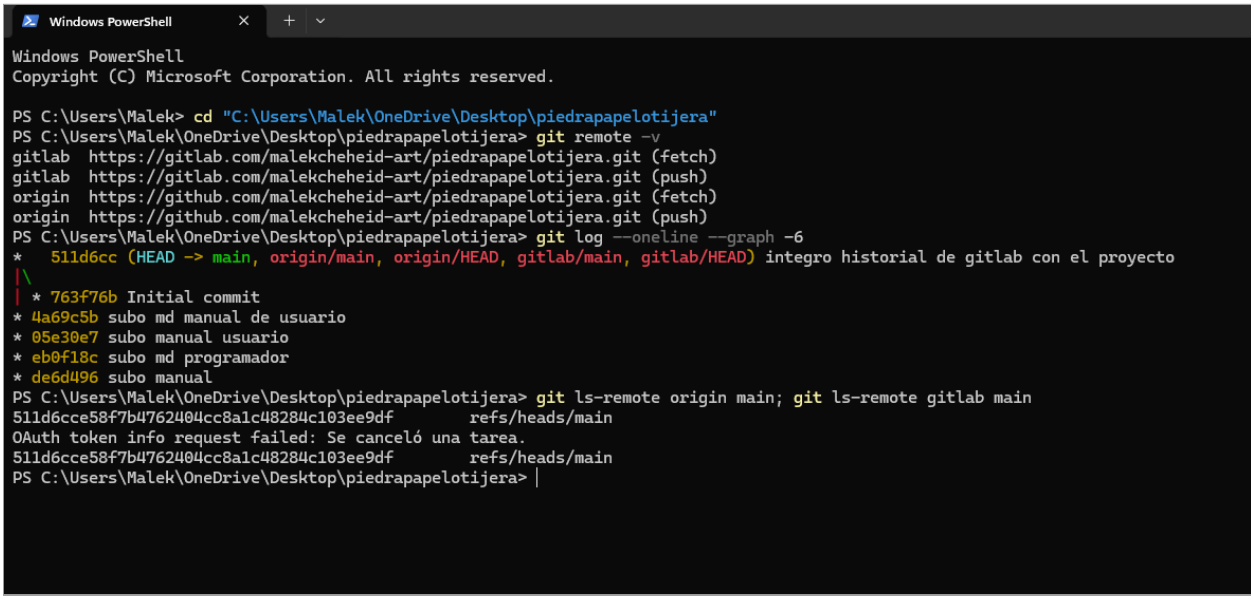


Figura 5. Verificación del dual push desde la consola.



Cuestionario técnico respondido

1. RAM vs. Disco: la RAM es volátil, por lo que al cerrar la consola se borran las variables del programa. El archivo en disco es no volátil y conserva las partidas, por eso la opción 2 sigue mostrando el historial.

2. Uso de ios::app: si usara ofstream sin esa bandera, el archivo se sobrescribiría y perdería las partidas anteriores. Con ios::app, cada registro se agrega al final.

3. cin.ignore(): limpia el carácter Enter pendiente en el búfer antes de llamar a getline, evitando que se salte la lectura del nombre.

4. struct: permite agrupar variables heterogéneas en una sola entidad, lo que simplifica pasar datos entre funciones y hace el código más claro.

5. Aplicación al Proyecto Integrador Anual: en el P.I.A. podría usar esta misma técnica para guardar logs de sensores, registros de usuarios, control de stock o fichajes de turnos en archivos de texto.

Conclusión

Logré cumplir con los objetivos de la actividad: usé struct para modelar cada partida y fstream para persistir los resultados en el disco. El programa conserva el historial entre ejecuciones y permite consultarlo en cualquier momento. Me sirvió para entender la persistencia de datos y cómo aplicarla en programas reales. Además, dejé el repositorio organizado con código, documentación, capturas y el informe.

Referencias

cppreference.com. (s. f.). *std::basic_ofstream::open*. En cppreference.com.

https://en.cppreference.com/w/cpp/io/basic_ofstream/open

GeeksforGeeks. (s. f.). *cin.ignore() function in C++*. GeeksforGeeks. https://www.geeksforgeeks.org/cpp/cin-ignore-function-in-cpp/

GeeksforGeeks. (s. f.). *Structs y estructuras de datos en C++*. GeeksforGeeks. https://www.geeksforgeeks.org/cpp/structures-in-cpp/

Código completo

```
/*
  ================================================================

  Materia: Laboratorio de Programacion (LPR) -- 5to Ano
  Institucion: E.E.S.T. N° 1 "Eduardo Ader" -- Vicente Lopez
  Archivo: src/main.cpp
  Actividad 7: Mini Juego Piedra, Papel o Tijera + Persistencia en Disco

  Autor: Malek Cheheid  (Grupo A)
  Docente: Yamil Ganduglia

  ================================================================
*/

#include <iostream>
#include <fstream>
#include <string>
#include <cstdlib>
#include <ctime>

using namespace std;

const string ARCHIVO_HISTORIAL = "src/historial_partidas.txt";

struct Partida {
    int idPartida;
    string jugador;
    string eleccionJugador;
    string eleccionIA;
    string resultado;
};
```

```
void jugarPartida(int& contadorID);
void guardarEnDisco(const Partida& p);
void leerHistorialDisco();
int cargarContadorID();

int main() {
    srand(time(0));

    int contadorID = cargarContadorID();
    int opcion = 0;

    cout << "=======================================================" << endl;
    cout << " LPR 2026 - ACTIVIDAD 7: PIEDRA, PAPEL O TIJERA " << endl;
    cout << " Estudiante / Grupo: Malek Cheheid (Grupo A) " << endl;
    cout << "=======================================================" << endl;

    do {
        cout << "\n--- MENU PRINCIPAL DE JUEGO Y PERSISTENCIA ---" << endl;
        cout << "1. Jugar Partida (Piedra, Papel o Tijera)" << endl;
        cout << "2. Ver Historial de Partidas Guardadas en Disco (.txt)" << endl;
        cout << "3. Salir del Sistema" << endl;
        cout << "Ingrese una opcion (1-3): ";
        if (!(cin >> opcion)) {
            cin.clear();
            cin.ignore(10000, '\n');
            opcion = 0;
        }

        if (opcion == 1) {
            jugarPartida(contadorID);
        } else if (opcion == 2) {
            leerHistorialDisco();
        } else if (opcion == 3) {
            cout << "\n[SISTEMA] Saliendo del programa... Datos resguardados en disco." << endl;
        } else {
            cout << "\n[ERROR] Opcion invalida. Reintente." << endl;
        }
    } while (opcion != 3);
```

```
    return 0;
}

void jugarPartida(int& contadorID) {
    Partida p;
    p.idPartida = contadorID++;

    cout << "\n--- NUEVA PARTIDA #" << p.idPartida << " ---" << endl;

    cin.ignore(); // Limpieza de bufer
    do {
        cout << "Ingrese su Nombre / Nickname: ";
        getline(cin, p.jugador);
        if (p.jugador.empty()) {
            cout << "[ERROR] El nombre no puede estar vacio." << endl;
        }
    } while (p.jugador.empty());

    int opcJugador = 0;
    do {
        cout << "\nElija su jugada:" << endl;
        cout << "1. Piedra" << endl;
        cout << "2. Papel" << endl;
        cout << "3. Tijera" << endl;
        cout << "Opcion (1-3): ";
        if (!(cin >> opcJugador)) {
            cin.clear();
            cin.ignore(10000, '\n');
            opcJugador = 0;
        }
        if (opcJugador < 1 || opcJugador > 3) {
            cout << "[ERROR] Opcion invalida. Reintente." << endl;
        }
    } while (opcJugador < 1 || opcJugador > 3);

    if (opcJugador == 1) p.eleccionJugador = "Piedra";
    else if (opcJugador == 2) p.eleccionJugador = "Papel";
    else p.eleccionJugador = "Tijera";
```

```
    int opcIA = (rand() % 3) + 1;
    if (opcIA == 1) p.eleccionIA = "Piedra";
    else if (opcIA == 2) p.eleccionIA = "Papel";
    else p.eleccionIA = "Tijera";

    cout << "\n[JUGADA] " << p.jugador << " eligio: " << p.eleccionJugador << endl;
    cout << "[JUGADA] La IA eligio: " << p.eleccionIA << endl;

    if (p.eleccionJugador == p.eleccionIA) {
        p.resultado = "Empate";
    } else if ((p.eleccionJugador == "Piedra" && p.eleccionIA == "Tijera") ||
               (p.eleccionJugador == "Papel" && p.eleccionIA == "Piedra") ||
               (p.eleccionJugador == "Tijera" && p.eleccionIA == "Papel")) {
        p.resultado = "Victoria";
    } else {
        p.resultado = "Derrota";
    }

    cout << "=> RESULTADO: " << p.resultado << "!" << endl;

    guardarEnDisco(p);
}

void guardarEnDisco(const Partida& p) {
    ofstream archivo(ARCHIVO_HISTORIAL, ios::app);

    if (!archivo.is_open()) {
        cerr << "[ERROR] No se pudo abrir el archivo para guardar la partida." << endl;
        return;
    }

    archivo << p.idPartida << ","
            << p.jugador << ","
            << p.eleccionJugador << ","
            << p.eleccionIA << ","
            << p.resultado << endl;

    archivo.close();
```

```
    cout << "[DISCO] Partida resguardada con exito en 'historial_partidas.txt'." << endl;
}

void leerHistorialDisco() {
    ifstream archivo(ARCHIVO_HISTORIAL);

    if (!archivo.is_open()) {
        cout << "\n[AVISO] Aun no existen partidas guardadas en disco." << endl;
        return;
    }

    cout << "\n========================================================" << endl;
    cout << " HISTORIAL DE PARTIDAS RECUPERADO DESDE DISCO " << endl;
    cout << "========================================================" << endl;

    string idStr, jugador, elJug, elIA, resultado;
    int total = 0;

    while (getline(archivo, idStr, ',') &&
           getline(archivo, jugador, ',') &&
           getline(archivo, elJug, ',') &&
           getline(archivo, elIA, ',') &&
           getline(archivo, resultado)) {
        total++;
        cout << "Partida #" << idStr
             << " | Jugador: " << jugador
             << " | " << elJug << " vs " << elIA
             << " | Resultado: " << resultado << endl;
    }

    archivo.close();

    if (total == 0) {
        cout << "[AVISO] El archivo de historial esta vacio." << endl;
    } else {
        cout << "\n[SISTEMA] Total de registros leidos desde disco: " << total << endl;
    }
    cout << "========================================================" << endl;
}
```

```
int cargarContadorID() {
    ifstream archivo(ARCHIVO_HISTORIAL);
    if (!archivo.is_open()) return 1;

    int maxId = 0;
    string idStr, jugador, elJug, elIA, resultado;
    while (getline(archivo, idStr, ',') &&
           getline(archivo, jugador, ',') &&
           getline(archivo, elJug, ',') &&
           getline(archivo, elIA, ',') &&
           getline(archivo, resultado)) {
        try {
            int id = stoi(idStr);
            if (id > maxId) maxId = id;
        } catch (...) {
            // Ignorar lineas mal formadas
        }
    }
    archivo.close();
    return maxId + 1;
}
```